ANUKRITI AI

Generative System Engineering & Digital Twin Platform for Medical Devices

Turning one sentence of plain English into a physics-simulated, deployable digital twin — small enough to run on a single-board computer.

Team Anukriti Ai

Vellore Institute of Technology, Chennai

Executive Summary

What the platform does, and what we measured

1

Describe

An engineer writes one sentence describing the device in plain English.

2

Generate

An LLM produces formal requirements, then synthesises the system architecture.

3

Simulate

Deterministic NumPy solvers model thermal, electrical, battery and fluid behaviour.

4

Deploy

A surrogate ML twin is trained on the simulation and serialised for inference.

Reference run — Smart Insulin Pump

7

requirements from one sentence

89.9%

mean channel accuracy

74 s

simulation to trained twin

742 KB

deployable model

Reference run: Smart Insulin Pump project on a laptop (WSL2 Ubuntu). Figures are measured, not projected.

The Problem

Medical device development takes 6 to 18 months — and the bottleneck is not manufacturing

Manual requirement engineering

Specifications are translated into formal requirements by hand — slow, inconsistent and error-prone.

Disconnected architecture modelling

Topology is drawn in separate tools, with no traceable link back to the requirements that justify it.

Late physics validation

Thermal and electrical behaviour is checked during physical prototyping — when correction is most expensive.

Manual regulatory mapping

IEC 60601-1 and ISO 13485 compliance is a separate documentation exercise, duplicating engineering effort.

The result: long cycles, expensive late-stage redesign, and a high barrier to entry for small MedTech teams.

One Automated Pipeline

Each stage feeds the next — no manual handoff between tools



1. **Traceable by construction**

   Every component records the requirement IDs it satisfies, so the audit trail is a by-product of the design, not a separate document.

2. **The physics is computed**

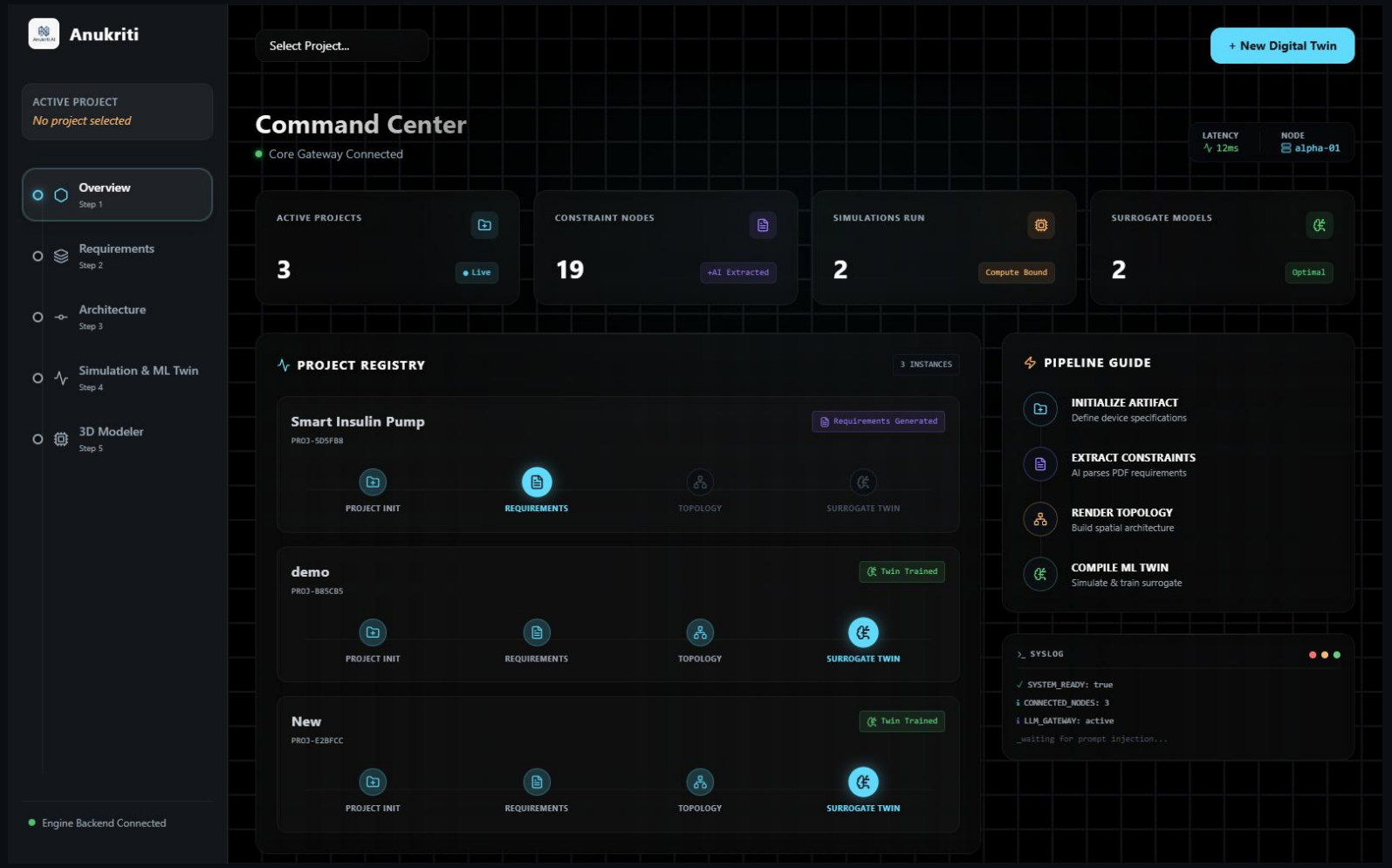
   Thermal, battery and leakage behaviour comes from numerical solvers — the LLM only chooses which models to run and with what parameters.

3. **Small enough for the edge**

   The full stack installs and serves from a Raspberry Pi 5. No cloud GPU or workstation is required.

The Platform in Use

Command centre — every device tracked through the pipeline



Project Registry

Each device moves through four stages: initialisation, requirements, topology and surrogate twin.

Live Stage Status

Stage badges update as each workflow step completes.

Guided Workflow

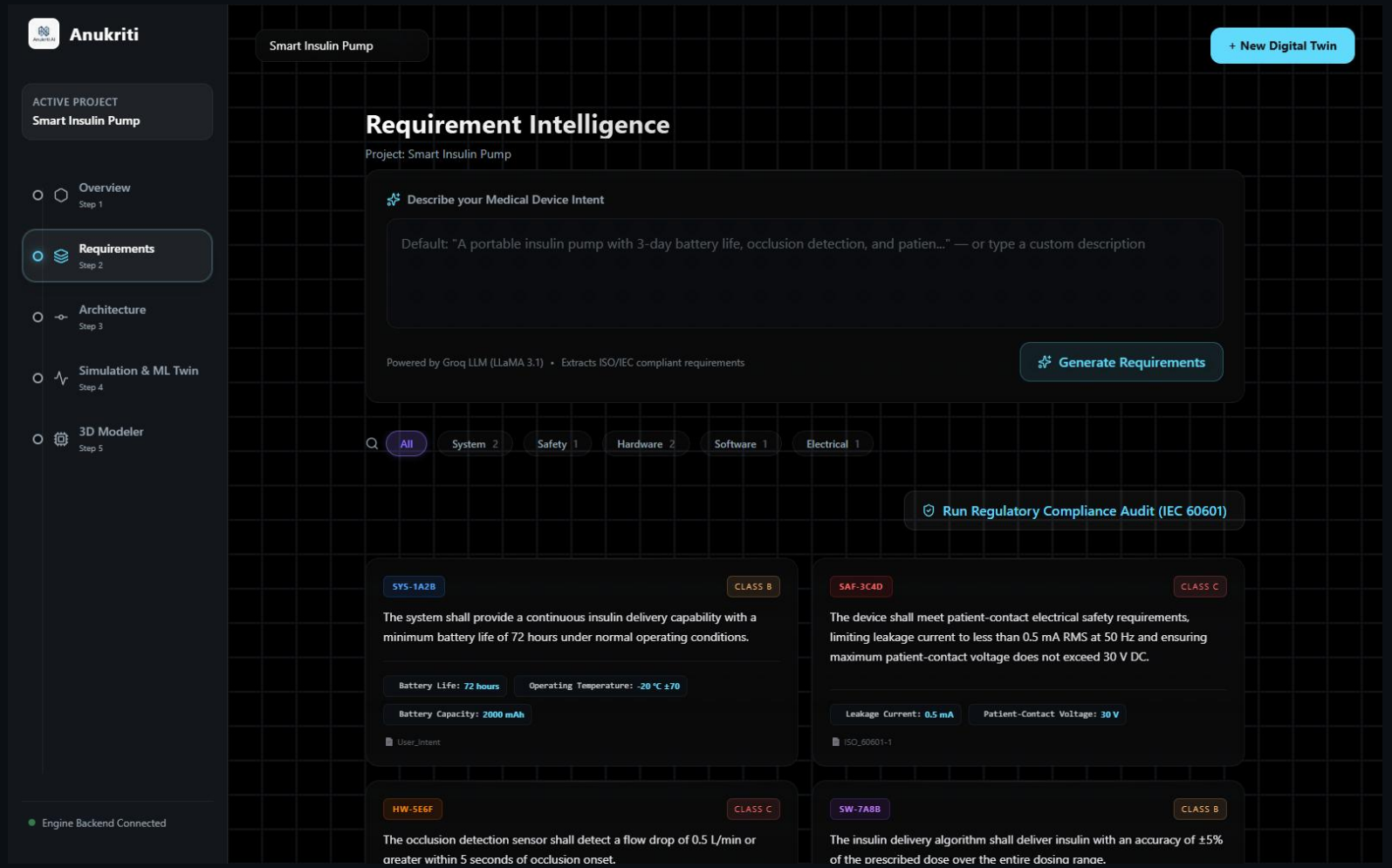
A step rail walks the engineer from requirements to twin in order.

Platform Statistics

Projects, extracted constraints, simulations and trained twins at a glance.

Requirements from One Sentence

Input: “A portable insulin pump with 3-day battery life, occlusion detection, and patient-contact electrical safety.”



7

formal requirements extracted from that single sentence.

Safety-classed

2 Class C and 5 Class B, aligned to IEC 62304 software safety classification.

Numeric constraints with units

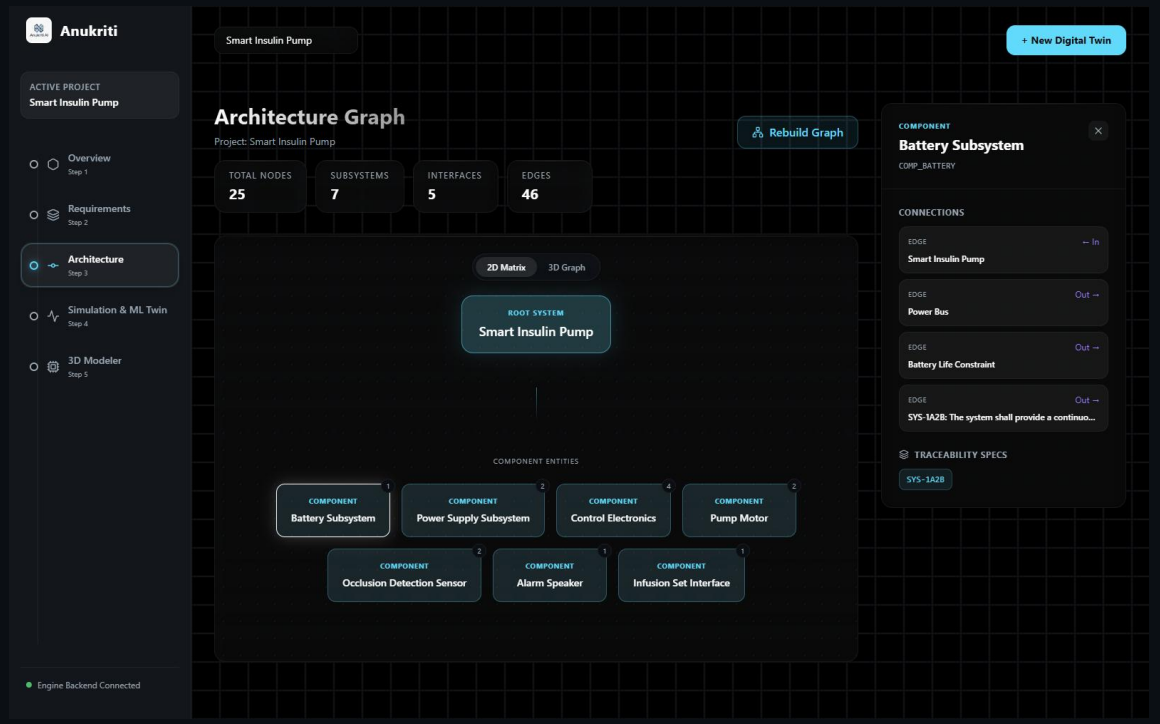
72 h battery life, 0.5 mA leakage, 5 s occlusion detection, 85 dB alarm.

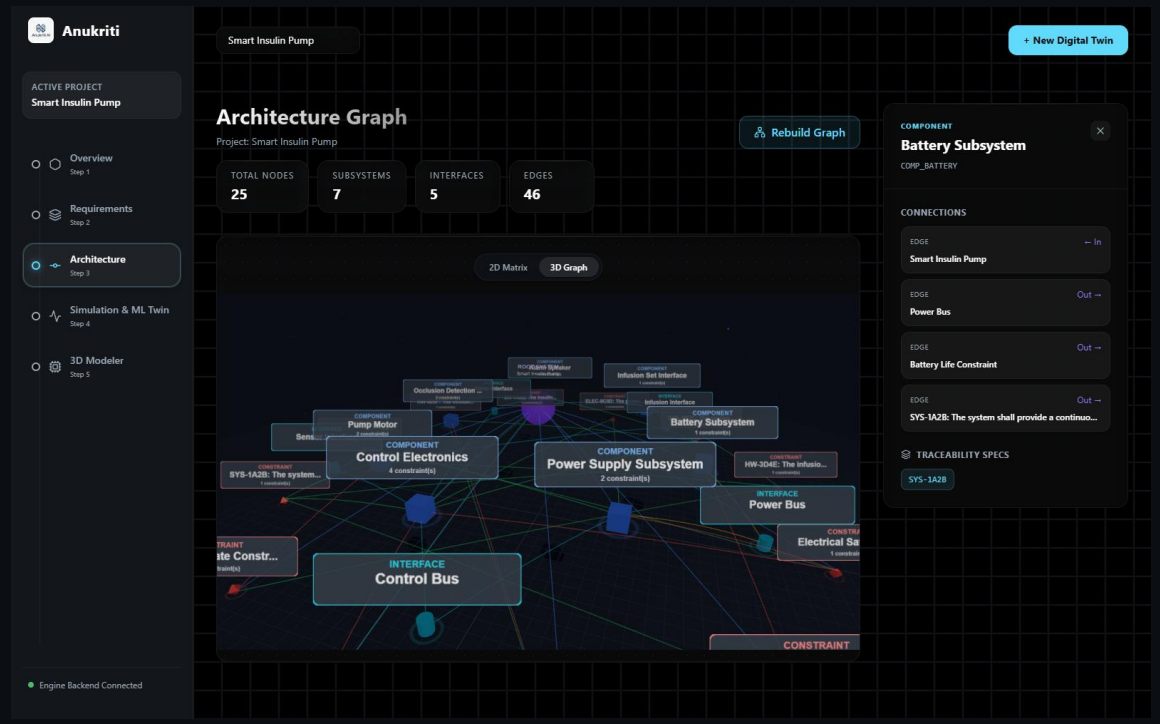
Linked to a source standard

IEC 60601-1, ISO 13485, IEC 62304 or the user's own intent.

Architecture Synthesis

7 subsystems, 5 interfaces and 46 traceability edges — derived from the requirements





2D matrix

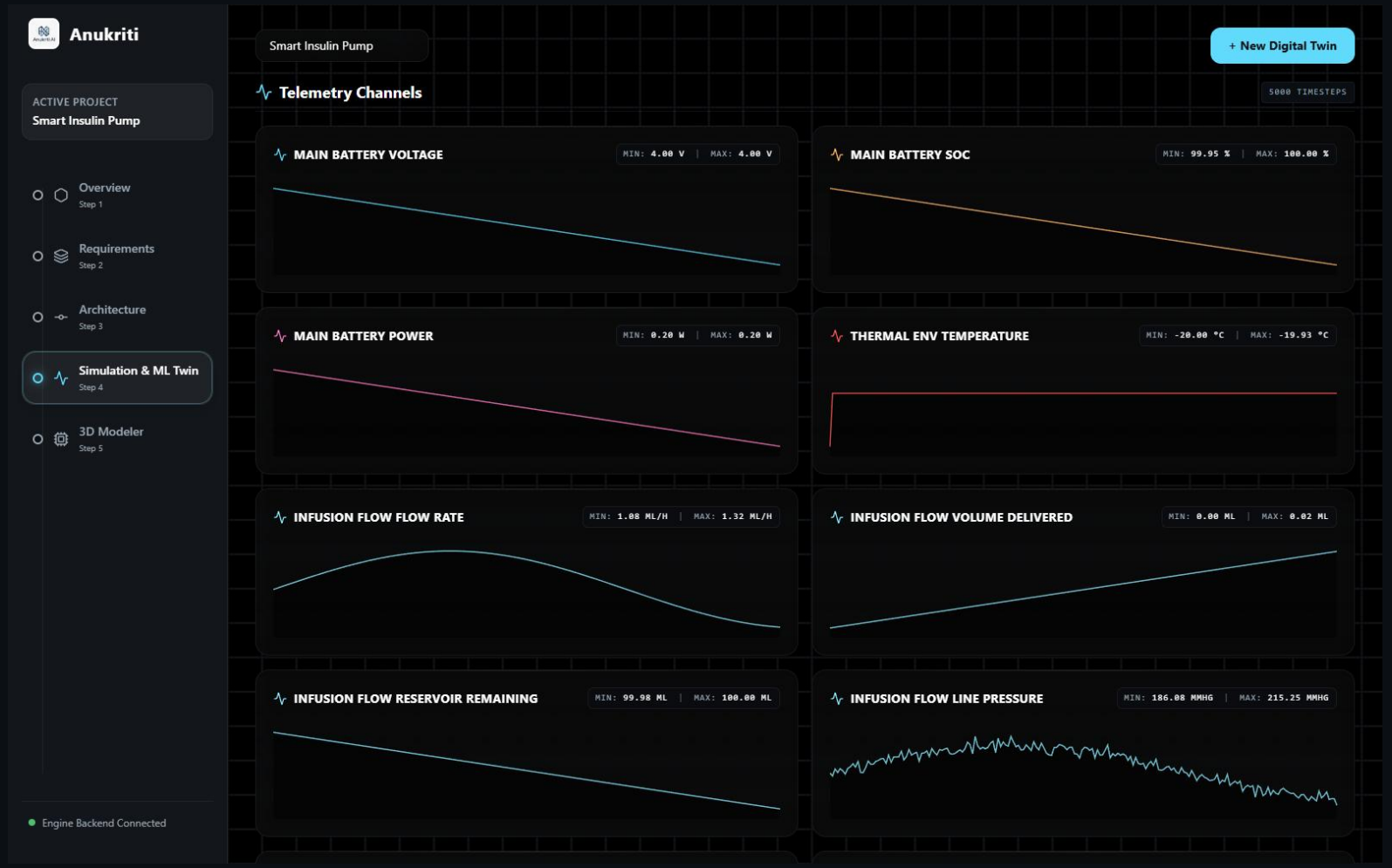
Selecting a subsystem opens its connections and the requirement IDs it satisfies — here, the Battery Subsystem traces to SYS-1A2B.

3D topology

The full graph of components, interfaces and constraints, rendered with Three.js and explorable in three dimensions.

Physics Simulation

Six numerical models selected for this device, solved over 7,200 time steps



LLM selects, NumPy solves

The model picks which solvers to run and their parameters; every value on screen comes from the solvers.

Battery

Shepherd discharge model — voltage, state of charge and power.

Infusion flow

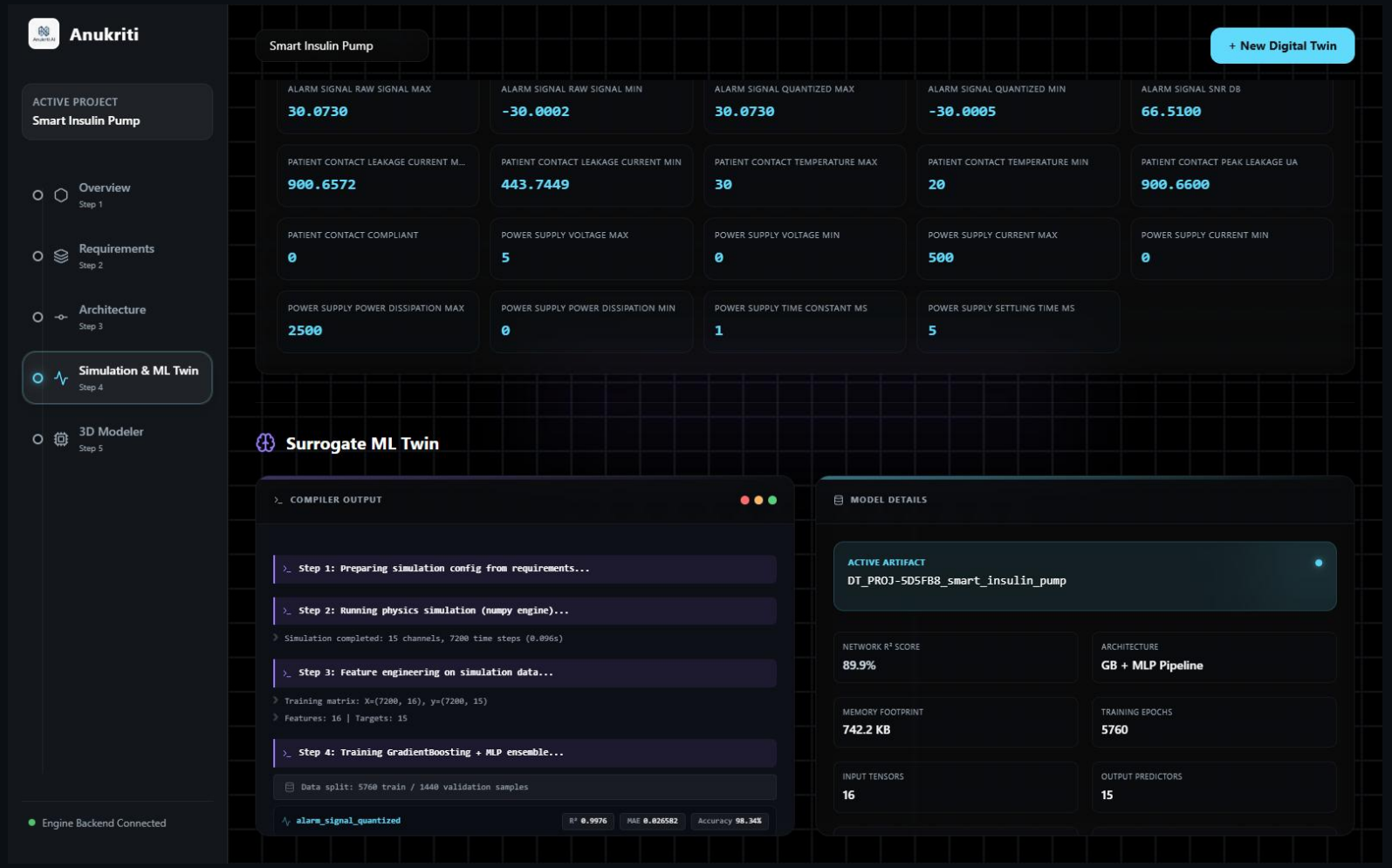
Peristaltic pulsation, delivered volume and noisy line pressure.

Limit checking

Modelled values are compared against requirement limits and breaches are flagged for review.

The Surrogate Digital Twin

GradientBoosting + MLP ensemble, trained on the simulation output



89.9%

mean per-channel accuracy (1 − normalised error) across 15 output channels.

742 KB

serialised model — compact enough for edge inference.

69 s

training time: 5,760 training and 1,440 validation samples, 16 features.

Healthcare Use Case

Design and verification of regulated, patient-contact medical devices

- **Infusion & insulin pumps**
- **Patient monitors**
- **Ventilators**
- **Portable diagnostics**

Early safety screening	Leakage current and thermal rise are modelled against requirement limits before any hardware exists.
Requirements traceability	Each component carries the requirement IDs it satisfies, producing the audit trail regulators expect.
Risk classification	Requirements are assigned safety class A, B or C, aligned to IEC 62304.
Faster time-to-patient	Shorter design cycles mean validated, life-critical devices reach patients sooner.
Lower barrier to entry	Runs on a single-board computer, opening device R&D to resource-constrained teams.

Scope: a design-stage engineering aid. It does not diagnose or treat, and is not itself a regulated device.

Three Computational Layers

Only the first is generative — the rest is deterministic mathematics

01

Language Model

Groq · openai/gpt-oss-20b

- Requirement extraction
- Architecture synthesis
- Simulation planning
- Compliance audit
- Enforced JSON output
- Deterministic fallbacks

02

Numerical Physics

NumPy · 8 solver models

- Shepherd battery model
- Newton cooling (Euler)
- RC step response
- Peristaltic flow
- DC motor back-EMF
- Leakage vs limit

03

Surrogate ML

scikit-learn ensemble

- GradientBoosting + MLP
- 16 engineered features
- One model per channel
- StandardScaler
- 80/20 validation
- joblib .pkl artefact

The Physics Engine

Eight numerical models — closed-form and numerically integrated, no machine learning

Battery discharge

Modified Shepherd model with polarisation and exponential zones; SoC tracking

Thermal

Newton's law of cooling with power dissipation, forward Euler integration

RC circuit

First-order step response, time constant and settling time

Fluid flow

Peristaltic pump with pulsation, cumulative volume and line pressure

Motor drive

DC motor with back-EMF, torque and inertia, integrated step-wise

Signal response

Drift, Gaussian noise, ADC quantisation and SNR

Leakage current

Insulation degradation with temperature coefficient, checked against a limit

Sensor response

First-order response with noise and saturation bounds

Limits are checked automatically; a breach raises a warning in the telemetry view.

Technology Stack

Open-source throughout, MIT licensed

Frontend

- Next.js 16.1.6
- React 19.2.3
- TypeScript 5
- Tailwind CSS 4
- Three.js 0.183
- Framer Motion 12

Backend

- FastAPI
- Uvicorn (ASGI)
- Pydantic 2
- NumPy 2
- scikit-learn
- NetworkX

AI & Infrastructure

- Groq inference API
- openai/gpt-oss-20b
- joblib serialisation
- Docker Compose
- Raspberry Pi OS
- Render / AWS EB

Repository: github.com/prayag-1771/Anukriti

Measured Outcomes

One complete run — device description to trained, serialised twin

89.9%

Mean channel accuracy

1 − normalised error, 15 channels

74 s

Simulation to twin

physics, training and serialisation

742 KB

Model footprint

compact enough for edge inference

7,200

Simulated samples

5,760 training · 1,440 validation

Requirements extracted from one sentence	**7 (2 Class C, 5 Class B)**
Architecture graph synthesised	**25 nodes, 46 edges**
Subsystems and interfaces identified	**7 subsystems, 5 interfaces**
Physics models selected and solved	**6**
Best single-channel fit	**R^2 0.9999**

Reference run: Smart Insulin Pump project on a laptop (WSL2 Ubuntu). Figures are measured, not projected.

Running at the Edge

Verified on device — the full stack installs and serves from one board

Board	**Raspberry Pi 5 Model B**
Processor	**4 x ARM Cortex-A76 (aarch64)**
Memory	**8 GB**
Operating system	**Raspberry Pi OS 64-bit (Debian 13)**
Runtime	**Python 3.13 · Node 20.19**
Cooling	**Official active cooler**
Power	**USB-C, 5 V**
Thermals	**49 °C with the platform running, no throttling**

Why edge matters

- Device telemetry and trained models stay on the premises.
- Only prompts and structured JSON reach the cloud LLM.
- The cost of entry is a single-board computer, not a simulation licence.
- Deployable in labs and clinics with limited connectivity.

Limitations & Roadmap

What the reference run taught us, and where the platform goes next

Observed limitations

- LLM-chosen simulation parameters need engineer review — in our run a 0.5 mA leakage limit was read as 0.5 µA.
- Short simulated windows left 10 of 15 channels near-constant, where R^2 is not informative.
- The surrogate learns one run's trajectory; it is not yet conditioned on operating parameters.
- Solvers are simplified and need validation against certified FEM/CFD before regulatory use.
- Project state is held in memory, and the compliance audit depends on an external LLM.

Roadmap

Near term

Unit-checked parameter validation between requirements and simulation inputs.

Near term

Parameter-conditioned surrogates, so one twin covers a design space, not a single run.

Mid term

Persistent project storage and an exportable Requirements Traceability Matrix.

Longer term

Validation against ANSYS or COMSOL on a reference device; on-device language models.

Months into minutes

Plain English to requirements, to architecture, to simulated physics, to a deployable machine-learning twin.

Team Anukriti Ai

Harsh Yadav
Team Leader

Avishkar Jaiswal
Team Member

Amol Bhatia
Team Member

Prayag Sharma
Team Member

Vellore Institute of Technology, Chennai

github.com/prayag-1771/Anukriti · **MIT Licence**